

Chain Rule & Backpropagation

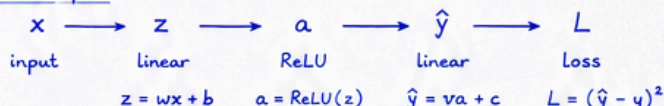
Page 1 of 2

Goal: Compute gradients of the loss L with respect to every parameter in a small neural network using the chain rule.

Key idea: During forward pass, information flows forward.
During backward pass, gradients flow backward!

1. A tiny neural network (one hidden unit)

Forward pass:



Our variables:

x : input (scalar) v, c : parameters of second layer
 w, b : parameters of first layer $\hat{y}$: prediction (scalar)
 z : pre-activation (scalar) y : true target (scalar)
 a : activation (scalar) L : loss (scalar)

2. Choose numerical values and do the forward pass

Let's pick:

$x = 2.0, y = 3.0, w = 1.0, b = -1.0, v = 2.0, c = 0.5$

Step 1: $z = wx + b = (1.0)(2.0) + (-1.0) = 1.0$

Step 2: $a = \text{ReLU}(z) = \max(0, 1.0) = 1.0$ (since $z > 0$)

Step 3: $\hat{y} = va + c = (2.0)(1.0) + 0.5 = 2.5$

Step 4: $L = (\hat{y} - y)^2 = (2.5 - 3.0)^2 = (-0.5)^2 = 0.25$

Forward pass summary: $z = 1.0, a = 1.0, \hat{y} = 2.5, L = 0.25$

3. What we want (gradients)

We need gradients of L with respect to all parameters:

$$\frac{dL}{dw}, \frac{dL}{db}, \frac{dL}{dv}, \frac{dL}{dc}$$

The loss L does not depend on w directly.

It depends on w through the chain:

$$w \longrightarrow z \longrightarrow a \longrightarrow \hat{y} \longrightarrow L$$

Therefore we use the chain rule.

4. Backpropagation (apply chain rule step by step)

We go backward.

① From L to $\hat{y}$: $\frac{dL}{d\hat{y}} = 2(\hat{y} - y)$ (because $L = (\hat{y} - y)^2$)

Numerically: $\frac{dL}{d\hat{y}} = 2(2.5 - 3.0) = -1.0$

② From $\hat{y}$ to a : $\frac{d\hat{y}}{da} = v$ (since $\hat{y} = va + c$)

Numerically: $\frac{d\hat{y}}{da} = v = 2.0$

Now multiply: $\frac{dL}{da} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{da} = (-1.0)(2.0) = -2.0$

③ From a to z : $\frac{da}{dz} = \begin{cases} 1, & \text{if } z > 0 \\ 0, & \text{if } z < 0 \end{cases}$ (ReLU derivative)

Here $z = 1.0 > 0 \Rightarrow \frac{da}{dz} = 1$

Now multiply: $\frac{dL}{dz} = \frac{dL}{da} \cdot \frac{da}{dz} = (-2.0)(1) = -2.0$

④ From z to w and b : $\frac{dz}{dw} = x, \frac{dz}{db} = 1$

Numerically: $\frac{dz}{dw} = x = 2.0, \frac{dz}{db} = 1.0$

Now multiply:

$$\frac{dL}{dw} = \frac{dL}{dz} \cdot \frac{dz}{dw} = (-2.0)(2.0) = -4.0$$

$$\frac{dL}{db} = \frac{dL}{dz} \cdot \frac{dz}{db} = (-2.0)(1.0) = -2.0$$

5. Gradients of v and c

We can also compute directly.

• For v : $\hat{y} = va + c \Rightarrow \frac{d\hat{y}}{dv} = a$

$$\frac{dL}{dv} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{dv} = (-1.0)(a) = (-1.0)(1.0) = -1.0$$

• For c : $\frac{d\hat{y}}{dc} = 1$

$$\frac{dL}{dc} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{dc} = (-1.0)(1) = -1.0$$

All gradients:

$$\frac{dL}{dw} = -4.0, \frac{dL}{db} = -2.0, \frac{dL}{dv} = -1.0, \frac{dL}{dc} = -1.0$$

6. Update parameters (gradient descent)

Choose learning rate $\eta = 0.1$

Rule: $\theta_{\text{new}} = \theta_{\text{old}} - \eta \frac{dL}{d\theta}$

$$w_{\text{new}} = 1.0 - 0.1(-4.0) = 1.4$$

$$b_{\text{new}} = -1.0 - 0.1(-2.0) = -0.8$$

$$v_{\text{new}} = 2.0 - 0.1(-1.0) = 2.1$$

$$c_{\text{new}} = 0.5 - 0.1(-1.0) = 0.6$$

Updated parameters:

$$w = 1.4, b = -0.8, v = 2.1, c = 0.6$$

7. Forward pass again with new parameters

Step 1: $z = wx + b = (1.4)(2.0) + (-0.8) = 1.2$

Step 2: $a = \text{ReLU}(z) = \max(0, 1.2) = 1.2$

Step 3: $\hat{y} = va + c = (2.1)(1.2) + 0.6 = 2.52 + 0.6 = 3.12$

Step 4: $L_{\text{new}} = (3.12 - 3.0)^2 = (0.12)^2 = 0.0144$

Loss decreased from 0.25 $\rightarrow$ 0.0144

Good! The parameters learned in the right direction. 😊

8. About ReLU and the "dying ReLU" problem

If in the forward pass we had $z \leq 0$, then ReLU gives $a = 0$.

Its derivative is $\frac{da}{dz} = 0$.

Then $\frac{dL}{dz} = \frac{dL}{da} \cdot \frac{da}{dz} = 0$.

Therefore: $\frac{dL}{dw} = \frac{dL}{dz} \cdot \frac{dz}{dw} = 0$ and $\frac{dL}{db} = 0$

No gradient flows through! The neuron cannot learn for that input. This is called the **dying ReLU** problem.

9. Implement it in NumPy (we write the backward pass)

```
# given: x, y
# parameters: w, b, v, c
# learning rate: eta

# ---- forward pass ----
z = w * x + b          # linear
a = max(0, z)          # ReLU
y_hat = v * a + c       # linear
loss = (y_hat - y)**2    # squared error

# ---- backward pass (backpropagation) ----
dl_dyhat = 2 * (y_hat - y)  # dL/dy_hat
dy_hat_da = v              # d(y_hat)/da
dl_da = dl_dyhat * dy_hat_da  # dL/da
da_dz = 1.0 if z > 0 else 0.0 # ReLU'
dl_dz = dl_da * da_dz      # dL/dz
dz_dw = x                  # dz/dw
dz_db = 1.0                # dz/db
dl_dw = dl_dz * dz_dw      # dL/dw
dl_db = dl_dz * dz_db      # dL/db
dl_dv = dl_dyhat * a       # dL/dv
dl_dc = dl_dyhat * 1.0     # dL/dc

# ---- gradient descent update ----
w = w - eta * dl_dw
b = b - eta * dl_db
v = v - eta * dl_dv
c = c - eta * dl_dc
```

This is exactly what autograd does for us automatically in PyTorch!

😊

Key takeaways:

- ★ Backpropagation uses the chain rule to compute gradients layer by layer.
- ★ Gradients flow backward: $L \rightarrow \hat{y} \rightarrow a \rightarrow z \rightarrow$ parameters.
- ★ Each gradient is a product of local derivatives.

- ★ If ReLU output is zero ($z \leq 0$), gradient cannot flow.
- ★ Once we know all gradients, we update parameters with gradient descent.

Understand the math deeply first, then the code becomes easy!

★